TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems — Clinical Operations

Kevin Kawchak
CEO, ChemicalQDevice
kevink@chemicalqdevice.com
ORCID: 0009-0007-5457-8667

March 5, 2026

DOI: 10.5281/zenodo.18870961

Keywords: Model Context Protocol, clinical trial operations, Physical AI, oncology robotics, multi-site trials, MCP servers, FHIR, DICOM, audit ledger, trial scalability, site onboarding, electronic data capture, eConsent

Abstract

Multi-site oncology clinical trials increasingly deploy autonomous robotic platforms for specimen handling, imaging guidance, and procedural automation. Integrating these Physical AI systems with clinical infrastructure—scheduling, electronic data capture (EDC), eConsent, imaging archives, and laboratory information systems—currently requires point-to-point integrations that grow quadratically with the number of platforms and sites. This paper presents TrialMCP, an open-source suite of five Model Context Protocol (MCP) servers that provides a standardized interoperability layer between autonomous trial robots and clinical systems. We describe the end-to-end operational workflow from token-based authentication through study status retrieval, imaging pointer acquisition, evidence logging, and provenance tracking. The architecture supports role-based access control with deny-by-default semantics across 23 MCP tools, HIPAA Safe Harbor de-identification, and SHA-256 hash-chained audit trails satisfying 21 CFR Part 11. Validation across 39 tests—spanning security, audit completeness, and integration scenarios—demonstrates operational readiness. We present deployment topology patterns for federated multi-site trials, an adoption pathway with three milestone phases, and quantitative success criteria including projected reductions in integration project count, site onboarding time, and audit preparation cycles. TrialMCP v0.2.0 establishes the M1 milestone: read-only clinical data servers with a complete authorization framework, positioning the system for production FHIR/DICOM proxy integration in subsequent milestones.

Contents

1 Introduction **3**
1.1 The Point-to-Point Integration Problem . . . 3
1.2 MCP as the Solution Layer . . . 3
1.3 Paper Scope and Contributions . . . 3

2 Methods **3**
2.1 Architecture Overview . . . 3
2.2 MCP Server Suite (v0.2.0) . . . 4
2.3 End-to-End Workflow Sequence . . . 4
2.4 Role-Based Access Control Model . . . 4
2.5 Scheduling, eConsent, and EDC Adjacency . . . 5
2.6 Synthetic Datasets . . . 5

3 Results **5**
3.1 Integration Project Reduction . . . 5
3.2 Validation Test Results . . . 5
3.3 Version Evolution . . . 6

3.4 Robot Platform Readiness Scores . . . 6
3.5 Operational Workflow Timing . . . 6

4 Discussion **7**
4.1 Why Point-to-Point Does Not Scale . . . 7
4.2 Deployment Topology for Federated Trials . . . 7
4.3 Adoption Pathway and Milestones . . . 7
4.4 Operational Scalability Metrics . . . 8
4.5 Scheduling, eConsent, and EDC Integration Patterns . . . 8
4.6 Comparison with Existing Integration Approaches . . . 8

5 Limitations and Future Work **8**
5.1 Current Limitations . . . 8
5.2 Future Work . . . 9

6 Conclusion **9**

1 Introduction

1.1 The Point-to-Point Integration Problem

Modern oncology clinical trials operate across 5–50 sites, each deploying distinct clinical systems: electronic health records (EHR), picture archiving and communication systems (PACS), EDC platforms, eConsent systems, scheduling engines, and laboratory information management systems (LIMS). When autonomous robotic platforms enter this ecosystem—cobots for specimen handling (Franka Emika Panda, USL 7.4), surgical robots for guided biopsies (da Vinci dVRK, USL 7.1), and assistive platforms for logistics (Kinova Gen3, USL 5.7)—each robot–system pair demands a bespoke integration [2].

For N robot platforms and M clinical systems, point-to-point integration produces $O(N \times M)$ connectors. A trial with 3 robot types and 6 clinical systems requires up to 18 separate integration projects, each with its own authentication flow, data mapping, error handling, and compliance audit scope. Adding a single new robot platform triggers 6 additional integration projects.

1.2 MCP as the Solution Layer

The Model Context Protocol (MCP), an open protocol under the Linux Foundation AAIF [4], provides a standardized interface for connecting AI tools to external data sources. TrialMCP leverages MCP to collapse the $O(N \times M)$ integration matrix into $O(N + M)$: each robot platform implements one MCP client, and each clinical system is fronted by one MCP server.

1.3 Paper Scope and Contributions

This paper presents the Clinical Operations perspective of TrialMCP, addressing:

1. The end-to-end workflow sequence for robot-agent clinical operations
2. Deployment topology for federated multi-site trials
3. Operational scalability analysis and integration project reduction
4. Adoption pathway with milestone-driven deliverables
5. Success criteria tied to cycle time, audit readiness, and site onboarding

The TrialMCP codebase (v0.2.0) is open-source and comprises 5 MCP servers, 23 tools, a reference robot agent client, synthetic datasets, and 39 passing tests [2].

2 Methods

2.1 Architecture Overview

TrialMCP implements a layered architecture with four tiers separating agent consumers from clinical data sources:

TrialMCP Layered Architecture

```
TIER 1: AGENT LAYER
+---------------------------+   +---------------------------+
| ROBOT AGENTS              |   | HUMAN AGENTS              |
| Franka Panda (USL 7.4)    |   | Trial Coordinators        |
| da Vinci dVRK  (USL 7.1)  |   | Data Monitors             |
| Kinova Gen3    (USL 5.7)  |   | Auditors                  |
+-------------|-------------+   +-------------|-------------+
              |                               |
TIER 2: AUTHORIZATION LAYER
+-------------------------------------------------------------+
| trialmcp-authz (Policy Decision Point)                      |
| Deny-by-Default RBAC | Token Management | Session Scoping |
+-------------------------------------------------------------+
              |
TIER 3: SERVICE LAYER
+----------------+ +------------------+ +-------------------+
| trialmcp-fhir  | | trialmcp-dicom   | | trialmcp-ledger   |
| 4 tools        | | 4 tools          | | 5 tools           |
| Read-Only FHIR | | Query/Retrieve   | | Audit Hash Chain  |
+----------------+ +------------------+ +-------------------+
              |
TIER 4: DATA PROVENANCE LAYER
+-------------------------------------------------------------+
| trialmcp-provenance (5 tools)                               |
```

```
| Source Registration | Lineage Tracking | SHA-256 Integrity |
+-------------------------------------------------------------+
```

2.2 MCP Server Suite (v0.2.0)

The five servers expose 23 MCP tools organized by clinical function:

Table 1: TrialMCP server suite: 23 tools across 5 servers (v0.2.0)

Server	Tools	Clinical Function
trialmcp-authz	5	Token issuance, RBAC evaluation, policy listing, revocation
trialmcp-fhir	4	Patient lookup, study status, resource read/search (de-identified)
trialmcp-dicom	4	C-FIND proxy, C-MOVE pointers, RECIST 1.1 measurements
trialmcp-ledger	5	Hash-chain append, verify, query, replay, chain status
trialmcp-provenance	5	Source registration, access recording, lineage, actor history
Total	**23**	

2.3 End-to-End Workflow Sequence

The reference robot agent (v0.2.0) executes a six-step workflow that exercises all five servers:

Robot Agent Workflow Sequence (6 Steps)

```
Robot Agent                        TrialMCP Servers
(Franka Panda)
     |
     |  Step 1: AUTHENTICATE
     |----------------------------> trialmcp-authz
     |   authz_issue_token           |
     |<----------------------------  | token_id + role
     |
     |  Step 2: FETCH TASK ORDER
     |----------------------------> trialmcp-fhir
     |   fhir_study_status           |
     |<----------------------------  | study phase + enrollment
     |
     |  Step 3: RETRIEVE IMAGING
     |----------------------------> trialmcp-dicom
     |   dicom_retrieve_pointer      |
     |<----------------------------  | retrieval_token + endpoint
     |
     |  Step 4: EXECUTE PROCEDURE
     |   (Robot performs clinical task)
     |
     |  Step 5: UPLOAD EVIDENCE
     |----------------------------> trialmcp-ledger
     |   ledger_append               |
     |<----------------------------  | audit_id + record_hash
     |
     |  Step 6: RECORD PROVENANCE
     |----------------------------> trialmcp-provenance
     |   provenance_record_access    |
     |<----------------------------  | record_id + confirmation
```

2.4 Role-Based Access Control Model

Four pre-configured roles govern access across all servers:

Table 2: RBAC permission matrix: role-to-server-tool mappings

Role	FHIR Access	DICOM Access	Ledger Access
trial_coordinator	All tools	All tools, all levels	Chain status + verify
robot_agent	fhir_read, study_status	query + retrieve (STUDY, SERIES)	Chain status + verify
data_monitor	fhir_search, study_status	query only (PATIENT, STUDY)	Chain status + verify
auditor	None	None	All tools

2.5 Scheduling, eConsent, and EDC Adjacency

TrialMCP v0.2.0 provides synthetic scheduling data demonstrating the integration pattern for adjacent clinical systems. The scheduling dataset includes 3 robotic procedures with:

- USL-scored platform assignments (Franka Panda 7.4, Kinova Gen3 5.7, dVRK 7.1)
- Prerequisite tracking (imaging completion, consent status, lab results)
- Cross-site coordination across 2 trial sites (Site A, Site B)

The provenance gateway (trialmcp-provenance) supports five source types that map to clinical adjacencies: `fhir_bundle`, `dicom_study`, `scheduling`, `esource`, and `econsent`. Each registered source receives lineage tracking and actor history throughout the trial lifecycle.

2.6 Synthetic Datasets

The validation suite operates on three synthetic datasets (SHA-256 checksums recorded in `datasets/manifest.json`):

Table 3: Synthetic datasets for development and validation

Dataset	Format	Records	Content
FHIR Bundle	FHIR R4	14	2 trials, 3 patients, conditions, observations
DICOM Index	Custom v1	3	CT, MR, PET/CT with RECIST 1.1 targets
Scheduling	Custom v1	3	Robotic procedures across 2 sites

3 Results

3.1 Integration Project Reduction

The MCP abstraction layer transforms the integration complexity from quadratic to linear:

Table 4: Integration project count: point-to-point vs. TrialMCP

Scenario	Robots	Point-to-Point	TrialMCP
Single-site pilot	1 robot, 6 systems	6 projects	7 (1 client + 6 servers)
Multi-site (3 robots)	3 robots, 6 systems	18 projects	9 (3 clients + 6 servers)
Scaled trial (7 robots)	7 robots, 6 systems	42 projects	13 (7 clients + 6 servers)
Adding 1 new robot	+1 robot	+6 projects	+1 project
Adding 1 new system	+1 system	$+N$ projects	+1 project

At 7 robot platforms, TrialMCP reduces integration projects by 69% (42 $\rightarrow$ 13). Each additional robot platform adds 1 project instead of 6.

3.2 Validation Test Results

The v0.2.0 test suite comprises 39 tests across three categories, all passing:

Table 5: Test suite results (v0.2.0): 39 tests, 100% pass rate

Category	Count	Coverage
Security	18	SSRF prevention (URL-based IDs rejected), injection defense, permission escalation, replay prevention, token expiry, health endpoints, policy decision traces
Audit Completeness	12	Every FHIR/DICOM/provenance tool call produces audit records, hash-chain integrity, genesis hash, replay ordering
Integration	9	End-to-end robot workflow, de-identification verification, permission enforcement, audit chain after workflows, cross-server trace
Total	**39**	Expanded from 30 (v0.1.0) to 39 (v0.2.0)

3.3 Version Evolution

The peer-review-driven development lifecycle has progressed through three releases:

Release Progression: v0.1.0 -¿ v0.1.1 -¿ v0.2.0

```
+------------------+     +------------------+     +------------------+
| v0.1.0 BUILD     |     | v0.1.1 REVIEW    |     | v0.2.0 HARDEN    |
|                  |     |                  |     |                  |
| 5 MCP servers    |---->| @codex audit     |---->| Shared error     |
| 23 tools         |     | Gap analysis     |     |   taxonomy       |
| 30 tests         |     | Recommendations  |     | Token expiry     |
| Reference client |     | Publication plan |     | Input validation |
| Synthetic data   |     |                  |     | 39 tests         |
+------------------+     +------------------+     +------------------+
```

Key improvements from v0.1.0 to v0.2.0:

- **Shared common module** (`servers/common/__init__.py`): Machine-readable error codes (`AUTHZ_DENIED`, `VALIDATION_FA` `NOT_FOUND`, `TOKEN_EXPIRED`)
- **Token expiry enforcement**: Tokens include `expires_at` timestamps; zero-TTL tokens rejected immediately
- **SSRF prevention**: URL patterns rejected in FHIR resource IDs and DICOM UIDs
- **Policy decision traces**: Every `authz_evaluate` response includes a trace of matching rules
- **Health endpoints**: All 5 servers expose `health` tool returning status and version
- **Pagination guards**: FHIR search enforces `max_results` to prevent unbounded result sets

3.4 Robot Platform Readiness Scores

The Unification Standard Level (USL) framework scores robot platforms on a 1.0–10.0 scale across four dimensions [3]:

Table 6: USL-scored robot platforms supported by TrialMCP

Platform	Type	USL	Band	Trial Role
Franka Emika Panda	Cobot	7.4	Advanced	Specimen handling, biopsy assist
da Vinci dVRK	Surgical	7.1	Advanced	Guided surgical procedures
Kinova Gen3	Cobot	5.7	Intermediate	Sample collection, logistics
Atlas Electric	Humanoid	5.8	Intermediate	Site logistics, equipment transport
Hugo RAS	Surgical	4.5	Foundational	Surgical assist (emerging)

3.5 Operational Workflow Timing

Each step in the six-step workflow produces quantifiable outputs:

Table 7: Workflow step outputs and audit artifacts

Step	Server	Tool	Output
1	trialmcp-authz	authz_issue_token	token_id, role, expires_at
2	trialmcp-fhir	fhir_study_status	study phase, enrollment count
3	trialmcp-dicom	dicom_retrieve_pointer	retrieval_token (3600s TTL)
4	(Robot execution)	—	Procedure completion
5	trialmcp-ledger	ledger_append	audit_id, SHA-256 record_hash
6	trialmcp-provenance	provenance_record_access	record_id, input/output hashes

4 Discussion

4.1 Why Point-to-Point Does Not Scale

The fundamental challenge of multi-site oncology trials is combinatorial integration complexity. Each trial site operates its own EHR (Epic, Cerner, Meditech), PACS (Orthanc, dcm4chee, commercial), EDC platform, and scheduling system. When Physical AI platforms enter this ecosystem, the number of integration surfaces multiplies.

Consider a Phase III trial spanning 20 sites with 3 robot types:

- **Point-to-point**: $3 \times 6 \times 20 = 360$ integration touchpoints (robot–system–site)
- **TrialMCP**: $3 + 6 = 9$ standard interfaces, deployed per-site with configuration

The protocol-level standardization eliminates per-site reimplementation of authentication flows, data mapping, error handling, and audit logging.

4.2 Deployment Topology for Federated Trials

TrialMCP supports a federated deployment model where each site runs its own server instances:

Federated Multi-Site Deployment Topology

```
   SITE A                  SITE B                  SITE C
+-----------------+    +-----------------+    +-----------------+
| trialmcp-fhir   |    | trialmcp-fhir   |    | trialmcp-fhir   |
| trialmcp-dicom  |    | trialmcp-dicom  |    | trialmcp-dicom  |
| trialmcp-authz  |    | trialmcp-authz  |    | trialmcp-authz  |
| trialmcp-ledger |    | trialmcp-ledger |    | trialmcp-ledger |
+--------|--------+    +--------|--------+    +--------|--------+
         |                      |                      |
         v                      v                      v
+-----------------------------------------------------------+
|        FEDERATED COORDINATION LAYER                       |
| Cross-Site Audit Sync | Data Harmonization | Privacy Budgets|
+-----------------------------------------------------------+
         |
         v
+-----------------------------------------------------------+
|      trialmcp-provenance (Central Gateway)                |
| Cross-Site Lineage | Actor History | Data Fingerprinting  |
+-----------------------------------------------------------+
```

Each site maintains data sovereignty while the federated coordination layer synchronizes audit logs and harmonizes vocabulary mappings (LOINC, RxNorm, MedDRA) across the trial network [1].

4.3 Adoption Pathway and Milestones

The three-phase adoption pathway progressively expands operational scope:

Table 8: Three-phase adoption pathway with milestone deliverables

Phase	Timeline	Status	Deliverables
M1	0–4 months	**Current**	Read-only FHIR/DICOM access, RBAC auth framework, audit ledger, 39 tests
M2	4–8 months	Planned	Production FHIR proxy (HAPI), DICOM proxy (Orthanc/dcm4chee), persistent audit storage, cross-site replay sync
M3	8–12 months	Planned	Reference robot agent with ROS 2, live USL platform demo, eConsent workflow, eSource upload with certified evidence chain

4.4 Operational Scalability Metrics

TrialMCP targets the following operational improvements:

- **Integration project reduction**: 69% fewer projects at 7 robot platforms (42 → 13)
- **Site onboarding**: Standardized MCP server deployment replaces per-site custom integrations; target is configuration-only onboarding within hours instead of weeks
- **Audit readiness**: Hash-chained ledger with replayable traces provides inspection-ready audit trails without manual log compilation
- **New platform onboarding**: Adding a new robot platform requires implementing one MCP client interface instead of M system-specific connectors

4.5 Scheduling, eConsent, and EDC Integration Patterns

While TrialMCP v0.2.0 focuses on read-only access, the architecture supports adjacent clinical system integration through the provenance gateway's source type taxonomy:

Clinical System Adjacency Model

```
Provenance Source Types       Adjacent Clinical Systems
+----------------------+      +---------------------------+
| fhir_bundle          | <--> | EHR (Epic, Cerner)        |
| dicom_study          | <--> | PACS (Orthanc, dcm4chee)  |
| scheduling           | <--> | Trial Scheduling Systems  |
| esource              | <--> | EDC / eSource Platforms    |
| econsent             | <--> | eConsent Systems           |
+----------------------+      +---------------------------+
```

Each source type registers with the provenance graph, enabling lineage tracking across system boundaries. Future M2 and M3 milestones will implement live connectors for these adjacencies.

4.6 Comparison with Existing Integration Approaches

Table 9: Integration approach comparison

Criterion	Point-to-Point	ESB/Middleware	TrialMCP (MCP)
Scalability	$O(N \times M)$	$O(N + M)$	$O(N + M)$
AI-native interface	No	No	Yes (MCP tools)
Built-in audit	No	Partial	Full hash-chain
Role-based access	Per-connector	Central	Deny-by-default RBAC
De-identification	Per-connector	Varies	Integrated pipeline
Open standard	No	Varies	Yes (Linux Foundation)

5 Limitations and Future Work

5.1 Current Limitations

1. **In-memory implementation**: All servers currently use in-memory data stores. Production deployments require persistent storage backends (PostgreSQL, MongoDB) and integration with actual FHIR servers (HAPI FHIR) and PACS systems (Orthanc, dcm4chee).

2. **Simplified token management**: The authorization framework uses simplified token management. Production deployments should integrate with OAuth2/OIDC identity providers (Keycloak, Auth0, Azure AD).
3. **Synthetic data only**: All datasets are synthetic. Real clinical data validation requires institutional partnerships, IRB approval, and patient consent.
4. **No write operations**: v0.2.0 is read-only. EDC uploads, eSource submissions, and eConsent status updates require controlled write operations planned for M2/M3.
5. **Single-process deployment**: The current architecture runs all servers in a single Python process. Production deployments will require containerized microservice orchestration (Kubernetes, Docker Compose).
6. **No digital signatures**: Audit records use SHA-256 hash chains but lack cryptographic signatures. W3C PROV alignment and PKI-based signing are planned for M2.

5.2 Future Work

- **M2 (4–8 months)**: Production FHIR server proxy (HAPI FHIR), DICOM proxy (Orthanc/dcm4chee), persistent audit storage with signed timestamps, cross-site replay log synchronization
- **M3 (8–12 months)**: Reference robot agent with ROS 2 integration, live demo with USL-scored platforms, eConsent workflow integration, eSource upload with certified evidence chain
- **Federated learning integration**: FedAvg, FedProx, and SCAFFOLD aggregation strategies for cross-site model training with differential privacy budgets [1]
- **CI/CD pipeline**: Automated testing, linting, and deployment workflows for continuous validation
- **Adversarial testing**: Fuzzing campaigns against MCP tool inputs, time-of-check/time-of-use (TOCTOU) race condition testing

6 Conclusion

TrialMCP addresses the fundamental scalability challenge of integrating Physical AI robotic platforms with clinical trial infrastructure. By standardizing the agent-to-system interface through the Model Context Protocol, the architecture reduces integration complexity from $O(N \times M)$ to $O(N + M)$—a 69% reduction in integration projects at scale. The v0.2.0 release establishes the M1 milestone with 5 MCP servers, 23 tools, deny-by-default RBAC, HIPAA Safe Harbor de-identification, SHA-256 hash-chained audit trails, and 39 passing tests across security, audit, and integration categories.

The federated deployment topology supports multi-site trials with per-site data sovereignty and cross-site audit synchronization. The three-phase adoption pathway provides a clear roadmap from read-only access (current) through production proxies (M2) to live robot integration (M3). For trial sponsors and CRO leadership, TrialMCP offers a standards-based path to robot/agent-ready clinical operations without the brittleness and cost of point-to-point integrations.

References

[1] Kevin Kawchak. Federated learning for PAI oncology trials. GitHub, 2026. Five-pillar architecture, six agentic AI patterns.

[2] Kevin Kawchak. Physical AI oncology trials platform. GitHub, 2026. Robots, software, connectivity, and USL framework.

[3] Kevin Kawchak. Unification standard level scoring for robotic platform evaluation. GitHub, 2026. USL oncology trials evaluation framework.

[4] Linux Foundation AAIF. Model context protocol specification. Linux Foundation, 2025. Open protocol for connecting AI tools to external data sources.

Acknowledgments

The author would like to acknowledge Anthropic for providing access to Claude Code Opus 4.6 for repository and paper generations; and OpenAI for providing access to GPT-5.2-Codex for peer-review recommendations, with ChatGPT 5.2 Thinking for project formulation and paper perspectives assistance.

Ethical Disclosures

The author of the article declares no competing interests.

Rights and Permissions

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author(s) and source are properly credited, a link to the Creative Commons license is provided, and any modifications made are indicated. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/.

Cite This Article

Kawchak K. TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems. *Zenodo*. 2026; 10.5281/zenodo.18870961.

TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems — Regulatory

Kevin Kawchak
CEO, ChemicalQDevice
kevink@chemicalqdevice.com
ORCID: 0009-0007-5457-8667

March 5, 2026

DOI: 10.5281/zenodo.18870961

Keywords: 21 CFR Part 11, ICH-GCP E6(R2), HIPAA Safe Harbor, audit trail, tamper-evident ledger, role-based access control, deny-by-default, compliance validation, MCP servers, clinical trial quality, electronic records, de-identification, computer system validation

Abstract

Autonomous robotic platforms in oncology clinical trials introduce novel regulatory challenges: every tool invocation by a robot agent constitutes an electronic record subject to 21 CFR Part 11, ICH-GCP E6(R2), and HIPAA privacy requirements. This paper presents the regulatory, quality, and compliance architecture of TrialMCP, a suite of five Model Context Protocol (MCP) servers designed with compliance-first principles. We detail the deny-by-default authorization model with four pre-configured roles, the HIPAA Safe Harbor de-identification pipeline implementing all 18 identifier categories, the SHA-256 hash-chained audit ledger providing tamper-evident records with replayable traces, and the machine-readable error taxonomy enabling structured incident classification. The validation approach encompasses 39 tests across security (SSRF prevention, injection defense, permission escalation, replay attack detection, token expiry enforcement), audit completeness (every tool call produces a signed record), and integration scenarios (end-to-end workflow verification with cross-server trace linkage). We map each TrialMCP control to its regulatory requirement, present the permission matrix by role, and describe the verification methodology that produces a defensible validation package. TrialMCP v0.2.0 demonstrates that MCP-based interoperability layers can be designed compliance-first, providing inspection-ready audit trails, least-privilege access enforcement, and tamper-evident electronic records for Physical AI clinical trial operations.

ableofcontents

1 Introduction

1.1 Regulatory Context for Physical AI in Clinical Trials

The introduction of autonomous robotic systems into oncology clinical trials creates a new class of regulated electronic records. When a Franka Emika Panda cobot (USL score 7.4) queries a DICOM imaging archive for pre-surgical planning, or a da Vinci dVRK surgical robot (USL 7.1) retrieves a patient's treatment protocol from a FHIR repository, each tool invocation constitutes an electronic record under 21 CFR Part 11 [3]. These records must be:

- **Attributable**: Linked to a specific actor (human or robot agent)
- **Legible**: Machine-readable and human-interpretable
- **Contemporaneous**: Timestamped at the time of creation
- **Original**: Preserved in their original form
- **Accurate**: Protected against unauthorized modification

These ALCOA principles, extended by ICH-GCP E6(R2) [1], govern all electronic records in clinical trials. TrialMCP implements these requirements at the protocol level, ensuring that compliance is inherent to the architecture rather than bolted on.

1.2 Compliance-First Design Philosophy

TrialMCP adopts a compliance-first design approach where regulatory requirements drive architectural decisions:

1. **Deny-by-default access**: No agent can access any tool without an explicit ALLOW rule
2. **De-identification by default**: All patient data passes through HIPAA Safe Harbor processing before return
3. **Audit by default**: Every tool invocation produces a signed audit record in the hash chain
4. **Tamper evidence by default**: SHA-256 hash chains detect any record modification or deletion
5. **Least privilege by default**: Roles are scoped to the minimum required tool access

1.3 Paper Scope

This paper presents the regulatory, quality, and compliance perspective of TrialMCP, addressing:

- Compliance claims and their regulatory basis
- Controls mapping from requirements to implementation
- Verification approach with test evidence
- Operational procedures for compliance officers and auditors
- Limitations, assumptions, and future compliance roadmap

2 Methods

2.1 Regulatory Requirements Mapping

TrialMCP maps each compliance control to its regulatory source:

Table 1: Regulatory requirements to TrialMCP controls mapping

Regulation	Requirement	TrialMCP Control
21 CFR 11.10(a)	System validation	39-test validation suite across 3 categories
21 CFR 11.10(b)	Readable copies	JSON-serialized audit records with replay
21 CFR 11.10(c)	Record protection	SHA-256 hash chain, append-only ledger
21 CFR 11.10(d)	Access control	RBAC with deny-by-default, 4 roles
21 CFR 11.10(e)	Audit trail	Hash-chained ledger with tamper detection
21 CFR 11.10(k)	Authority checks	`authz_evaluate` with policy decision traces
ICH-GCP 5.5.3	Electronic records	Complete audit trail for all MCP tool calls
HIPAA 164.514	Safe Harbor	18-identifier de-identification pipeline
ISO 14971	Risk management	Permission policies, error taxonomy
IEC 80601-2-77	Robot safety	Safety-constrained execution patterns

2.2 Audit Hash Chain Architecture

The trialmcp-ledger implements a SHA-256 hash-chained audit ledger satisfying 21 CFR Part 11 electronic records requirements:

Audit Hash Chain Structure

```
+------------------+     +------------------+     +------------------+
|  GENESIS BLOCK   |     |  AUDIT RECORD 1  |     |  AUDIT RECORD 2  |
|                  |     |                  |     |                  |
|  prev: 000...0   |---->|  prev: hash(G)   |---->|  prev: hash(R1)  |
|  server: system  |     |  server: fhir    |     |  server: dicom   |
|  tool: init      |     |  tool: fhir_read |     |  tool: dicom_qry |
|  caller: system  |     |  caller: robot-1 |     |  caller: robot-1 |
|  hash: SHA-256   |     |  hash: SHA-256   |     |  hash: SHA-256   |
+------------------+     +------------------+     +------------------+
```

Each `AuditRecord` contains seven fields that are canonically serialized (JSON, sorted keys) and hashed:

Audit Record Hash Computation (v0.2.0)

```
def _compute_hash(self) -> str:
```

```
    content = json.dumps({
        "audit_id": self.audit_id,
        "timestamp": self.timestamp,
        "server": self.server,
        "tool": self.tool,
        "caller_id": self.caller_id,
        "parameters": self.parameters,
        "result_summary": self.result_summary,
        "previous_hash": self.previous_hash,
    }, sort_keys=True)
    return hashlib.sha256(content.encode()).hexdigest()
```

Chain properties:

- **Append-only**: Records cannot be modified or deleted after creation
- **Hash-linked**: Each record includes the SHA-256 hash of its predecessor
- **Tamper-evident**: Any modification to any record breaks the chain
- **Replayable**: Full trace can be replayed for incident review
- **Verifiable**: `ledger_verify` recomputes all hashes and validates all links

2.3 HIPAA Safe Harbor De-identification Pipeline

The trialmcp-fhir server implements HIPAA Safe Harbor de-identification (45 CFR 164.514(b)) [2] with the following controls:

Table 2: HIPAA Safe Harbor 18-identifier de-identification controls

#	Identifier	TrialMCP Action
1	Names	Removed from Patient resources
2	Geographic data	Address field removed
3	Dates	Birth date generalized to year only
4	Phone numbers	Telecom field removed
5	Fax numbers	Telecom field removed
6	Email addresses	Telecom field removed
7	SSN	Identifier field removed
8	Medical record numbers	Pseudonymized via HMAC-SHA256
9	Health plan IDs	Identifier field removed
10	Account numbers	Identifier field removed
11	Certificate/license	Identifier field removed
12	Vehicle identifiers	Not applicable (removed if present)
13	Device identifiers	Not applicable (removed if present)
14	URLs	Identifier field removed
15	IP addresses	Not stored
16	Biometric identifiers	Not applicable (removed if present)
17	Full-face photos	Not applicable (removed if present)
18	Other unique IDs	Identifier field removed

The pseudonymization function uses HMAC-SHA256 with a site-specific salt:

HMAC-SHA256 Pseudonymization (v0.2.0)

```
def pseudonymize_id(self, identifier: str) -> str:
    h = hashlib.sha256(
        f"{self._salt}:{identifier}".encode()
    )
    return f"pseudo-{h.hexdigest()[:16]}"
```

Subject references in clinical resources (Condition, Observation, MedicationAdministration, AdverseEvent) are also pseudonymized, ensuring referential consistency across de-identified resources.

2.4 Authorization Policy Engine

The trialmcp-authz server implements a policy decision point (PDP) with deny-by-default semantics:

Authorization Decision Flow

```
Agent Request
     |
     v
+--------------------+
| Match Policy Rules |  (role, server, tool)
+--------------------+
     |
     +--- Any DENY match? ---> DENY (explicit deny precedence)
     |
     +--- Any ALLOW match? --> ALLOW (with trace)
     |
     +--- No match? ---------> DENY (deny-by-default)
     |
     v
+--------------------+
| Return Decision    |
| + Matching Trace   |  (rule_id, effect for each match)
+--------------------+
```

The engine returns a policy decision trace with every evaluation, enabling auditors to verify the exact rules that contributed to each authorization decision.

2.5 Permission Matrix by Role

Table 3: Complete permission matrix: role × server × tool access

Role	Server	Tools	Effect
trial_coordinator	trialmcp-fhir	All (wildcard)	ALLOW
trial_coordinator	trialmcp-dicom	All (wildcard)	ALLOW
robot_agent	trialmcp-fhir	fhir_read, fhir_study_status	ALLOW
robot_agent	trialmcp-dicom	dicom_query, dicom_retrieve_pointer	ALLOW
data_monitor	trialmcp-fhir	fhir_search, fhir_study_status	ALLOW
data_monitor	trialmcp-dicom	dicom_query	ALLOW
data_monitor	trialmcp-provenance	provenance_register_source	DENY
auditor	trialmcp-ledger	All (wildcard)	ALLOW
All roles	trialmcp-ledger	ledger_chain_status, ledger_verify	ALLOW

Nine default policy rules are loaded at server initialization. The `deny-provenance-write` rule demonstrates explicit DENY precedence: even if a future ALLOW rule were added for `data_monitor` on `trialmcp-provenance`, the explicit DENY would take priority.

2.6 Error Taxonomy

TrialMCP v0.2.0 introduces a shared error taxonomy with machine-readable error codes (introduced in the `servers/common` module):

Table 4: Machine-readable error codes for structured incident classification

Error Code	Description
`AUTHZ_DENIED`	Authorization denied by policy engine
`VALIDATION_FAILED`	Input validation failure (SSRF prevention, format check)
`NOT_FOUND`	Requested resource does not exist
`INTERNAL_ERROR`	Unexpected server error
`TOKEN_EXPIRED`	Session token past expiration timestamp
`TOKEN_REVOKED`	Session token explicitly revoked
`PERMISSION_DENIED`	Role lacks permission for requested operation
`INVALID_INPUT`	Malformed or unrecognized input
`RATE_LIMITED`	Request rate limit exceeded

2.7 Input Validation and SSRF Prevention

Three validation functions in `servers/common/__init__.py` protect against injection and SSRF:

Input Validation Functions (v0.2.0)

```
_URL_PATTERN = re.compile(r"https?://", re.IGNORECASE)
_DICOM_UID_PATTERN = re.compile(r"^[\d.]+$")
_FHIR_ID_PATTERN = re.compile(r"^[A-Za-z0-9\-._]+$")

def validate_fhir_id(value: str) -> bool:
    if _URL_PATTERN.search(value):
        return False
    return bool(_FHIR_ID_PATTERN.match(value))

def validate_dicom_uid(value: str) -> bool:
    if _URL_PATTERN.search(value):
        return False
    return bool(_DICOM_UID_PATTERN.match(value))
```

3 Results

3.1 Verification Test Evidence

The v0.2.0 validation suite produces 39 test results across three categories:

3.1.1 Security Tests (18 tests)

Table 5: Security test evidence: 18 tests, all passing

Test Class	Tests	Count
SSRF Prevention	URL in FHIR resource ID rejected; encoded URL variant rejected; URL in DICOM UID rejected	3
Injection Prevention	JSON injection in FHIR search; SQL injection in ledger append	2
Permission Escalation	Auditor denied SERIES query; monitor denied DICOM retrieve; unknown role denied; DENY overrides ALLOW	4
Replay Prevention	Hash chain integrity; token revocation; token expiry enforcement	3
Health Endpoints	FHIR, DICOM, authz, ledger health status	4
Policy Traces	Allow decision with trace; deny decision with trace	2
Total		**18**

3.1.2 Audit Completeness Tests (12 tests)

Table 6: Audit completeness test evidence: 12 tests, all passing

Test Class	Tests	Count
FHIR Audit	fhir_read, fhir_search, patient_lookup, study_status	4
DICOM Audit	dicom_query produces audit; denied requests produce audit	2
Ledger Integrity	10-record chain integrity; genesis hash verification; replay ordering	3
Provenance Audit	register_source, record_access, verify_integrity	3
Total		**12**

3.1.3 Integration Tests (9 tests)

Table 7: Integration test evidence: 9 tests, all passing

Test Class	Tests	Count
Robot Workflow	Authentication; task order fetch; imaging retrieval; evidence upload; full 5-step workflow	5
Verification	Audit chain after workflow; de-identification in responses; permission enforcement; cross-server trace linkage	4
Total		**9**

3.2 De-identification Verification

The integration test `test_deidentification_in_workflow` verifies HIPAA Safe Harbor compliance on Patient resources:

De-identification Verification Test

```
def test_deidentification_in_workflow(self) -> None:
    result = self.fhir.handle_tool_call(
        "fhir_read",
        {"resource_type": "Patient",
         "resource_id": "patient-001"}
    )
    patient = result["resource"]
    assert "name" not in patient      # Name removed
    assert "address" not in patient   # Address removed
    assert patient["birthDate"] == "1965"  # Year only
    assert patient["id"].startswith("pseudo-")  # Pseudonymized
```

3.3 Chain Integrity Verification

The `ledger_verify` tool recomputes every hash in the chain and validates all predecessor links:

Chain Verification Process

```
  For each record R[i] in the chain:
    1. Recompute SHA-256 hash of R[i]'s content
    2. Compare with stored record_hash
       -> Mismatch indicates record tampering
    3. Compare R[i].previous_hash with R[i-1].record_hash
       -> Mismatch indicates chain link breakage
    4. For R[0]: verify previous_hash == GENESIS (64 zeros)

  Result: { valid: true/false, chain_length: N, errors: [...] }
```

3.4 Token Lifecycle Security

Token management enforces the complete lifecycle:

Token Lifecycle State Machine

```
  ISSUED -----> VALID -----> EXPIRED
    |                          ^
    |                          | (expires_at reached)
    +--------> REVOKED         |
               (explicit       |
                revocation)    |
                               |
  Zero-TTL tokens transition directly:
  ISSUED -----> EXPIRED (immediate rejection)
```

The v0.2.0 test suite includes a zero-TTL token test that verifies immediate expiry enforcement:

Token Expiry Test (v0.2.0)

```
def test_token_expiry_enforcement(self) -> None:
    server = AuthzMCPServer()
    token = server.handle_tool_call(
        "authz_issue_token",
        {"caller_id": "test", "role": "robot_agent",
         "expires_seconds": 0}
    )
    result = server.handle_tool_call(
        "authz_validate_token",
        {"token_id": token["token_id"]}
    )
    assert result["valid"] is False
```

3.5 Cross-Server Trace Completeness

The integration test `test_cross_server_trace` verifies end-to-end audit linkage:

Cross-Server Audit Trace Verification

```
Workflow:  authz -> fhir -> dicom -> ledger -> provenance
              |        |        |        |           |
              v        v        v        v           v
Ledger:   [auth]   [fhir]   [dicom]  [evidence]  [prov]
            |        |        |        |           |
            +--------+--------+--------+-----------+
                            |
                   All linked by SHA-256
                   hash chain in ledger
```

The test confirms that the ledger contains records from both `trialmcp-fhir` and `trialmcp-dicom` servers, and that the full chain remains valid after a complete workflow execution.

4 Discussion

4.1 21 CFR Part 11 Compliance Analysis

TrialMCP's audit ledger addresses five key 21 CFR Part 11 requirements:

1. **11.10(a) System validation**: The 39-test suite provides evidence of system validation, covering security controls, audit completeness, and integration correctness. The test expansion from 30 (v0.1.0) to 39 (v0.2.0) demonstrates ongoing validation maintenance.
2. **11.10(c) Record protection**: The append-only, hash-chained ledger prevents record modification or deletion. Any tampering is detected by `ledger_verify`, which recomputes all hashes.
3. **11.10(d) Access control**: Deny-by-default RBAC with four roles and nine policy rules ensures that access is explicitly granted. The permission matrix is auditable via `authz_list_policies`.
4. **11.10(e) Audit trail**: Every MCP tool call across all five servers produces a signed audit record. The `ledger_replay` tool generates sequential traces for compliance review.
5. **11.10(k) Authority checks**: The `authz_evaluate` tool checks role-server-tool authorization before every operation. Policy decision traces document the exact rules that contributed to each decision.

4.2 ICH-GCP Traceability

ICH-GCP E6(R2) [1] requires that electronic records maintain complete traceability. TrialMCP achieves this through:

- **Actor identification**: Every audit record includes `caller_id` identifying the agent (human or robot)
- **Timestamp precision**: ISO 8601 UTC timestamps on all records
- **Operation detail**: Server name, tool name, and parameters captured in each record
- **Result tracking**: `result_summary` field captures the outcome of each operation
- **Chain linkage**: SHA-256 hash chain provides ordering and integrity guarantees

4.3 Defensible Validation Package

The TrialMCP validation package comprises:

Validation Package Contents

```
VALIDATION PACKAGE STRUCTURE
============================
1. Test Suite (39 tests)
   +-- tests/security/test_ssrf_injection.py     (18 tests)
   +-- tests/audit/test_audit_completeness.py    (12 tests)
   +-- tests/integration/test_robot_workflow.py   (9 tests)

2. Regulatory Mapping (Table 1)
   +-- 21 CFR Part 11 controls
   +-- ICH-GCP E6(R2) requirements
   +-- HIPAA Safe Harbor identifiers
```

```
3. Policy Documentation
   +-- 9 default policy rules (code-as-documentation)
   +-- Permission matrix (Table 5)
   +-- Error taxonomy (Table 6)

4. Dataset Governance
   +-- datasets/manifest.json (SHA-256 checksums)
   +-- datasets/README.md (data dictionary)

5. Peer-Review Evidence
   +-- peer-review/trialmcp_pack_v0.1.1_peer_review.md
   +-- peer-review/v0.2.0_peer_review_response.md
```

4.4 Data Flow Security Model

The complete data flow from agent request to de-identified response traverses five security layers:

Privacy-Preserving Data Flow (5 Layers)

```
Agent Request
     |
Layer 1: TOKEN PRESENTATION
     | Token validated (not expired, not revoked)
     v
Layer 2: AUTHORIZATION (deny-by-default)
     | authz_evaluate(role, server, tool)
     | Check ALLOW rules, check DENY overrides
     v
Layer 3: INPUT VALIDATION
     | FHIR ID format, DICOM UID format
     | URL rejection (SSRF prevention)
     v
Layer 4: DE-IDENTIFICATION
     | HIPAA Safe Harbor (18 identifiers)
     | HMAC-SHA256 pseudonymization
     v
Layer 5: AUDIT LOGGING
     | SHA-256 hash chain append
     | 21 CFR Part 11 compliance
     v
De-identified Response
```

4.5 Inspection Readiness Assessment

TrialMCP provides the following inspection readiness capabilities:

- **Trace completeness**: Every tool call across all 23 tools is logged with full context
- **Replay capability**: `ledger_replay` generates chronologically ordered traces from any timestamp
- **Integrity verification**: `ledger_verify` confirms no record has been modified, deleted, or reordered
- **Policy transparency**: `authz_list_policies` returns all 9 active policy rules as structured data
- **Decision auditability**: Every authorization decision includes a trace of matching rules
- **Health monitoring**: All 5 servers expose health endpoints for operational status verification

5 Limitations and Future Work

5.1 Current Limitations

1. **No cryptographic signatures**: Audit records use SHA-256 hash chains for tamper detection but lack PKI-based digital signatures. Full 21 CFR Part 11 electronic signature compliance requires integration with certificate authorities.
2. **In-memory storage**: The audit ledger is in-memory. Production deployments require persistent, write-once storage (append-only databases, immutable object storage) with backup and recovery.

3. **No W3C PROV alignment**: The provenance model uses a custom schema. Future versions should align with the W3C PROV standard for interoperability with external provenance systems.
4. **Simplified identity management**: Token-based authentication without OAuth2/OIDC integration. Production systems require enterprise identity provider integration.
5. **No adversarial testing**: The test suite covers known attack vectors but does not include fuzzing, property-based testing, or TOCTOU race condition scenarios.
6. **Synthetic data validation only**: All tests use synthetic data. Validation with real clinical data patterns (de-identified) is needed for production readiness.
7. **No signed timestamps**: Timestamps are server-generated without external time authority verification. RFC 3161 timestamping would strengthen temporal non-repudiation.

5.2 Future Compliance Work

- **PKI-based signatures**: Digital signatures on audit records using X.509 certificates
- **W3C PROV alignment**: Provenance records conforming to the PROV-DM data model
- **OAuth2/OIDC integration**: Enterprise identity provider support (Keycloak, Azure AD)
- **Adversarial testing**: Fuzzing campaigns, property-based testing, TOCTOU scenarios
- **GxP qualification**: IQ/OQ/PQ documentation for regulated deployment environments
- **Part 11 assessment toolkit**: Self-assessment checklist for sites deploying TrialMCP

6 Conclusion

TrialMCP demonstrates that MCP-based interoperability layers for Physical AI clinical trial operations can be designed compliance-first. The v0.2.0 release provides deny-by-default authorization with four pre-configured roles, HIPAA Safe Harbor de-identification implementing all 18 identifier categories, SHA-256 hash-chained audit trails with replayable traces, and a machine-readable error taxonomy for structured incident classification. The 39-test validation suite—covering SSRF prevention, injection defense, permission escalation, replay attacks, token expiry enforcement, audit completeness, and cross-server trace linkage—produces a defensible validation package suitable for regulatory inspection.

The compliance architecture ensures that every robot agent tool invocation constitutes a traceable, tamper-evident electronic record, satisfying the ALCOA principles required by 21 CFR Part 11 and ICH-GCP E6(R2). For QA/CSV teams, auditors, and compliance officers, TrialMCP offers inspection readiness through complete audit traces, least-privilege access enforcement, and policy decision transparency.

References

[1] ICH. ICH E6(R2) guideline for good clinical practice. International Council for Harmonisation, 2016.

[2] U.S. Department of Health and Human Services. HIPAA privacy rule: Safe harbor method for de-identification. HHS, 2012.

[3] U.S. Food and Drug Administration. 21 CFR part 11: Electronic records; electronic signatures. Code of Federal Regulations, 2003.

Acknowledgments

The author would like to acknowledge Anthropic for providing access to Claude Code Opus 4.6 for repository and paper generations; and OpenAI for providing access to GPT-5.2-Codex for peer-review recommendations, with ChatGPT 5.2 Thinking for project formulation and paper perspectives assistance.

Ethical Disclosures

The author of the article declares no competing interests.

Rights and Permissions

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author(s) and source are properly credited, a link to the Creative Commons license is provided, and any modifications made are indicated. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/.

Cite This Article

Kawchak K. TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems. *Zenodo*. 2026; 10.5281/zenodo.18870961.

TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems — Interoperability

Kevin Kawchak
CEO, ChemicalQDevice
kevink@chemicalqdevice.com
ORCID: 0009-0007-5457-8667

March 5, 2026

DOI: 10.5281/zenodo.18870961

Keywords: healthcare interoperability, FHIR R4, DICOM, Model Context Protocol, MCP servers, HL7, IHE, PACS integration, clinical data exchange, agent-tool interface, federated deployment, HAPI FHIR, Orthanc, dcm4chee, provenance gateway, role-scoped capabilities

Abstract

Healthcare interoperability in multi-site oncology clinical trials requires composing established standards—FHIR R4 for clinical data, DICOM for medical imaging—with emerging AI agent protocols. This paper presents TrialMCP from the healthcare interoperability perspective, positioning the Model Context Protocol (MCP) as the agent-tool interface that cleanly composes with FHIR and DICOM in federated, multi-site clinical environments. We describe the capability model of five MCP servers exposing 23 tools organized by clinical domain: read-only FHIR access with integrated de-identification (4 tools), DICOM query/retrieve gating with role-based permission enforcement (4 tools), audit chain-of-custody with hash-linked integrity (5 tools), authorization policy evaluation with deny-by-default semantics (5 tools), and data provenance tracking with lineage graphs (5 tools). The architecture defines clean boundaries between the MCP protocol layer, healthcare data standards (FHIR R4, DICOM), and clinical systems (EHR, PACS), with implementation pathways through HAPI FHIR, Orthanc, and dcm4chee. We present the server and tool definitions, security gate architecture, deployment topology for federated trials, vocabulary harmonization patterns (LOINC, RxNorm, MedDRA), and conformance considerations for HL7 and IHE integration profiles. Validation across 39 tests demonstrates predictable interfaces with audit hooks at every boundary. TrialMCP v0.2.0 reduces integration complexity by replacing $N \times M$ point-to-point connectors with $N + M$ standardized interfaces, providing clean interoperability boundaries for Physical AI agents in regulated clinical environments.

ableofcontents

1 Introduction

1.1 The Interoperability Challenge

Multi-site oncology clinical trials operate at the intersection of three standards ecosystems:

1. **Clinical data standards**: FHIR R4 [1] for structured clinical data (patients, research studies, observations, adverse events)
2. **Medical imaging standards**: DICOM [5] for imaging data (C-FIND queries, C-MOVE retrieval, study/series/instance hierarchy)
3. **AI agent protocols**: Model Context Protocol (MCP) [4] for connecting AI tools to external data sources

The challenge is composing these standards without creating a monolithic integration layer. Each standard serves a different purpose: FHIR defines clinical data representation, DICOM defines imaging data exchange, and MCP defines agent-tool interaction. TrialMCP positions MCP as the *agent-facing interface* that cleanly mediates access to FHIR and DICOM resources.

1.2 MCP as the Agent-Tool Interface

The Model Context Protocol, standardized under the Linux Foundation AAIF, defines a protocol for AI tools to connect to external data sources. In the TrialMCP architecture, MCP serves as the boundary between:

- **AI agents** (robot platforms, LLM-based planners, monitoring systems) that consume clinical data
- **Clinical systems** (EHR, PACS, EDC, scheduling) that produce and manage clinical data

This separation produces clean interfaces: agents speak MCP, clinical systems speak FHIR/DICOM, and TrialMCP servers translate between them with authorization, de-identification, and audit at every boundary.

1.3 Paper Scope

This paper addresses the Healthcare Interoperability & Standards perspective:

- Capability model and tool surface area per server
- Read-only FHIR pattern with integrated de-identification
- DICOM query/retrieve gating with permission enforcement
- Provenance gateway concept for cross-system lineage
- Integration patterns with HAPI FHIR, Orthanc, and dcm4chee
- Deployment topology for federated multi-site trials
- Conformance considerations for HL7 and IHE profiles

2 Methods

2.1 Capability Model

TrialMCP defines five MCP servers, each with a distinct capability boundary:

Capability Model: 5 Servers, 23 Tools

```
+-------------------------------------------------------------------+
|                    MCP CAPABILITY SURFACE                         |
+-------------------------------------------------------------------+
|                                                                   |
|  trialmcp-authz        trialmcp-fhir        trialmcp-dicom      |
|  [Policy Decision]     [Clinical Data]      [Imaging Data]      |
|  +-----------------+   +-----------------+  +-----------------+ |
|  | authz_evaluate  |   | fhir_read       |  | dicom_query     | |
|  | authz_issue_    |   | fhir_search     |  | dicom_retrieve_ | |
|  |   token         |   | fhir_patient_   |  |   pointer       | |
|  | authz_validate_ |   |   lookup        |  | dicom_study_    | |
|  |   token         |   | fhir_study_     |  |   metadata      | |
|  | authz_list_     |   |   status        |  | dicom_recist_   | |
|  |   policies      |   +-----------------+  |   measurements  | |
|  | authz_revoke_   |                        +-----------------+ |
|  |   token         |                                            |
|  +-----------------+                                            |
|                                                                  |
|  trialmcp-ledger        trialmcp-provenance                     |
|  [Audit Chain]          [Data Lineage]                          |
|  +-----------------+   +-----------------+                      |
|  | ledger_append   |   | provenance_     |                      |
|  | ledger_verify   |   |   register_src  |                      |
|  | ledger_query    |   | provenance_     |                      |
|  | ledger_replay   |   |   record_access |                      |
|  | ledger_chain_   |   | provenance_     |                      |
|  |   status        |   |   get_lineage   |                      |
|  +-----------------+   | provenance_     |                      |
|                        |   get_actor_hst |                      |
|                        | provenance_     |                      |
|                        |   verify_intgty |                      |
|                        +-----------------+                      |
+-------------------------------------------------------------------+
```

2.2 Server and Tool Definitions

Each MCP server exposes tools with typed input schemas following the MCP tool specification:

Table 1: Tool input schema patterns across TrialMCP servers

Server	Tool	Required Input Parameters
trialmcp-fhir	fhir_read	resource_type (enum, 9 types), resource_id (string)
trialmcp-fhir	fhir_search	resource_type (enum), params (object, optional)
trialmcp-fhir	fhir_patient_lookup	patient_id (string)
trialmcp-fhir	fhir_study_status	study_id (string)
trialmcp-dicom	dicom_query	query_level (enum), caller_role (enum)
trialmcp-dicom	dicom_retrieve_pointer	study_instance_uid (string), caller_role (enum)
trialmcp-dicom	dicom_study_metadata	study_instance_uid (string), caller_role (enum)
trialmcp-dicom	dicom_recist_measurements	study_instance_uid (string)
trialmcp-ledger	ledger_append	server, tool, caller_id, result_summary (strings)
trialmcp-ledger	ledger_verify	(none)
trialmcp-ledger	ledger_query	server, tool, caller_id, since, limit (all optional)
trialmcp-ledger	ledger_replay	since (optional)
trialmcp-ledger	ledger_chain_status	(none)
trialmcp-authz	authz_evaluate	role, server, tool (strings)
trialmcp-authz	authz_issue_token	caller_id, role (strings)
trialmcp-authz	authz_validate_token	token_id (string)
trialmcp-authz	authz_list_policies	(none)
trialmcp-authz	authz_revoke_token	token_id (string)
trialmcp-provenance	provenance_register_source	source_type (enum), origin_server, description
trialmcp-provenance	provenance_record_access	source_id, action (enum), actor_id, actor_role, tool_call, input_data
trialmcp-provenance	provenance_get_lineage	source_id (string)
trialmcp-provenance	provenance_get_actor_history	actor_id (string)
trialmcp-provenance	provenance_verify_integrity	data, expected_hash (strings)

2.3 Read-Only FHIR Pattern

The trialmcp-fhir server implements a read-only access pattern with integrated de-identification:

FHIR Read-Only Access Pattern with De-identification

```
MCP Client                trialmcp-fhir              FHIR Data Store
     |                         |                          |
     | fhir_read(Patient,      |                          |
     |   "patient-001")        |                          |
     |------------------------>|                          |
     |                    [1] Validate FHIR ID            |
     |                    (reject URLs, enforce format)   |
     |                         |                          |
     |                    [2] Read raw resource            |
     |                         |------------------------->|
     |                         |<-------------------------|
     |                         | Raw Patient resource     |
     |                    [3] De-identify                  |
     |                    - Remove: name, address, telecom |
     |                    - Generalize: birthDate -> year  |
     |                    - Pseudonymize: id -> pseudo-xxx |
     |                         |                          |
     |                    [4] Emit audit record            |
     |                         |                          |
     | De-identified Patient   |                          |
     |<------------------------|                          |
```

Supported FHIR R4 resource types (9 types relevant to oncology trials):

- Patient
- ResearchStudy
- ResearchSubject
- Condition
- MedicationAdministration
- Observation
- AdverseEvent
- DiagnosticReport
- ImagingStudy

2.4 DICOM Query/Retrieve Gating

The trialmcp-dicom server acts as a DICOM gateway with role-based permission enforcement:

Table 2: DICOM permission gating by role

Role	Query Levels	Modalities	Retrieve
trial_coordinator	PATIENT, STUDY, SERIES	CT, MR, PT, DX, CR	Yes
robot_agent	STUDY, SERIES	CT, MR, RTSTRUCT, RTPLAN	Yes
data_monitor	PATIENT, STUDY	CT, MR, PT	No
auditor	STUDY only	(none)	No

The gating model ensures that:

- Robot agents access only imaging modalities relevant to their clinical tasks (CT, MR for guidance; RTSTRUCT, RTPLAN for treatment planning)
- Data monitors can query but not retrieve pixel data
- Auditors have minimal visibility (STUDY-level metadata only) sufficient for audit purposes
- Retrieval tokens are time-limited (3600-second TTL) and scoped to specific studies

2.5 Provenance Gateway Concept

The trialmcp-provenance server functions as a data lineage gateway that tracks every access and transformation across the system:

Provenance Gateway: Cross-System Lineage Tracking

```
Data Source Registration          Access Recording
+-----------------------+         +---------------------------+
| Source Types:          |         | Actions:                  |
| - fhir_bundle         |         | - read                    |
| - dicom_study         |         | - transform               |
| - scheduling          |         | - export                  |
| - esource             |         | - deidentify              |
| - econsent            |         | - aggregate               |
+-----------------------+         +---------------------------+
         |                                   |
         v                                   v
+-----------------------------------------------------------+
|              PROVENANCE GRAPH (DAG)                       |
|                                                           |
|  Source --[read]--> Record --[transform]--> Record        |
|    |                 |                       |          |
|    +-- actor_id       +-- input_hash          +-- output|
|    +-- origin_server  +-- tool_call              _hash  |
|    +-- metadata       +-- actor_role                     |
+-----------------------------------------------------------+
         |                                   |
         v                                   v
+----------------------------+   +----------------------------+
| Lineage Queries            |   | Integrity Verification     |
| get_lineage(source_id)     |   | verify_integrity(data,     |
| get_actor_history(actor)   |   |   expected_hash)           |
+----------------------------+   +----------------------------+
```

The provenance gateway records SHA-256 fingerprints for both input and output data, enabling post-hoc verification that data has not been modified in transit.

2.6 Security Gate Architecture

Every MCP tool call traverses multiple security gates before accessing clinical data:

Security Gate Architecture (5 Gates)

```
MCP Tool Call
     |
GATE 1: TOKEN VALIDATION
     | authz_validate_token (not expired, not revoked)
     |
GATE 2: AUTHORIZATION POLICY
     | authz_evaluate (role + server + tool)
     | Deny-by-default with explicit DENY precedence
     |
GATE 3: INPUT VALIDATION
     | validate_fhir_id / validate_dicom_uid
     | URL rejection (SSRF prevention)
     | Format enforcement (alphanumeric / numeric-dot)
     |
GATE 4: DATA DE-IDENTIFICATION
     | HIPAA Safe Harbor (18 identifiers)
     | HMAC-SHA256 pseudonymization
     |
GATE 5: AUDIT LOGGING
     | ledger_append (SHA-256 hash chain)
     | provenance_record_access (lineage)
     |
Secured Response
```

2.7 Implementation Pathway: HAPI FHIR, Orthanc, dcm4chee

TrialMCP v0.2.0 uses in-memory data stores. The architecture defines clear integration pathways to production systems:

Table 3: Production integration pathways for TrialMCP servers

TrialMCP Server	Production Backend	Integration Pattern
trialmcp-fhir	HAPI FHIR [7]	OAuth2-scoped REST client to HAPI FHIR R4 API; de-identification applied at the MCP layer before return
trialmcp-dicom	Orthanc [6] or dcm4chee	DICOM networking (C-FIND/C-MOVE) or DICOMweb (WADO-RS/QIDO-RS) proxy; permission gating at MCP layer
trialmcp-ledger	PostgreSQL or MongoDB	Append-only table/collection with hash chain; write-once storage semantics
trialmcp-authz	Keycloak / Azure AD	OAuth2/OIDC token exchange; policy rules persisted in external policy store
trialmcp-provenance	Graph database (Neo4j)	Provenance records as graph edges; W3C PROV-DM alignment

2.8 Vocabulary Harmonization

Multi-site trials require consistent terminology across sites. TrialMCP supports vocabulary harmonization through:

- **LOINC**: Laboratory observation identifiers for lab results standardization
- **RxNorm**: Medication terminology for drug administration records
- **MedDRA**: Medical dictionary for adverse event coding and regulatory reporting
- **CTCAE v5.0**: Common Terminology Criteria for Adverse Events grading
- **RECIST 1.1**: Response Evaluation Criteria in Solid Tumors for imaging assessment

The FHIR data store supports search parameters that map to these vocabularies, enabling cross-site queries with consistent coding.

3 Results

3.1 Tool Surface Area Analysis

The 23-tool surface area distributes across five functional domains:

Table 4: Tool surface area distribution by functional domain

Domain	Server	Tools	Interface Pattern
Authorization	trialmcp-authz	5	Token lifecycle + policy evaluation
Clinical Data	trialmcp-fhir	4	Read-only FHIR R4 with de-identification
Medical Imaging	trialmcp-dicom	4	DICOM C-FIND/C-MOVE proxy with gating
Audit Trail	trialmcp-ledger	5	Append-only hash chain with replay
Data Lineage	trialmcp-provenance	5	Source registration + access tracking
Total	**5 servers**	**23**	

3.2 FHIR Resource Coverage

The trialmcp-fhir server provides access to 9 FHIR R4 resource types covering the oncology trial lifecycle:

Table 5: FHIR R4 resource type coverage with synthetic dataset counts

Resource Type	Count	Clinical Purpose
Patient	3	Demographics (de-identified, year-only birth date)
ResearchStudy	2	Phase II robotic biopsy, Phase III immunotherapy
ResearchSubject	3	Enrollment status linking patients to studies
Condition	2	NSCLC, colorectal cancer diagnoses
Observation	2	Tumor size measurements, biomarker values
MedicationAdministration	1	Pembrolizumab treatment record
AdverseEvent	1	Grade 2 fatigue (CTCAE v5.0)
DiagnosticReport	—	Supported type (no synthetic data)
ImagingStudy	—	Supported type (no synthetic data)

3.3 DICOM Study Coverage

The synthetic DICOM index contains 3 imaging studies with RECIST 1.1 measurements:

Table 6: DICOM study index with RECIST 1.1 target lesion tracking

Study	Modality	Series	Targets	Site
CT Chest	CT	2	TL-001, TL-002	Site A
MR Abdomen	MR	3	TL-001	Site B
PET/CT Whole Body	PT	2	TL-001, TL-002	Site A

3.4 Integration Complexity Reduction

The MCP abstraction transforms integration complexity:

Integration Complexity: Before and After MCP

```
BEFORE (Point-to-Point):
+--------+     +--------+     +--------+
| Robot  |---->| EHR    |     | Robot  |---->| PACS   |
| Type A |     |        |     | Type A |     |        |
+--------+     +--------+     +--------+     +--------+
+--------+     +--------+     +--------+     +--------+
| Robot  |---->| EHR    |     | Robot  |---->| PACS   |
| Type B |     |        |     | Type B |     |        |
+--------+     +--------+     +--------+     +--------+
Connectors: N x M = 2 x 2 = 4 (grows quadratically)

AFTER (TrialMCP):
+--------+                                   +--------+
| Robot  |---> +------------------+ -------->| EHR    |
| Type A |     | trialmcp-fhir    |          +--------+
+--------+     | trialmcp-dicom   |          +--------+
+--------+     | trialmcp-authz   | -------->| PACS   |
| Robot  |---> | trialmcp-ledger  |          +--------+
| Type B |     | trialmcp-prov    |
```

```
+--------+     +------------------+
Connectors: N + M = 2 + 2 = 4 (grows linearly)
```

At small scale the absolute counts are similar, but at trial scale (7 robots, 6 systems), point-to-point requires 42 connectors while TrialMCP requires 13.

3.5 Cross-Server Transaction Patterns

The end-to-end robot workflow demonstrates four cross-server transaction patterns:

Table 7: Cross-server transaction patterns in the robot workflow

Step	Primary Server	Audit Server	Transaction Pattern
1	trialmcp-authz	(self-audited)	Token issuance with role binding
2	trialmcp-fhir	trialmcp-ledger	De-identified clinical data read
3	trialmcp-dicom	trialmcp-ledger	Gated imaging retrieval pointer
5	trialmcp-ledger	(self-appended)	Evidence record with hash chain
6	trialmcp-provenance	(self-audited)	Lineage record with SHA-256 fingerprints

3.6 Federated Deployment Topology

The federated deployment model supports multi-site trials with per-site data sovereignty:

Federated Multi-Site Deployment with Data Harmonization

```
     SITE A                    SITE B                    SITE C
+------------------+     +------------------+     +------------------+
| trialmcp-fhir    |     | trialmcp-fhir    |     | trialmcp-fhir    |
| -> HAPI FHIR     |     | -> HAPI FHIR     |     | -> HAPI FHIR     |
| trialmcp-dicom   |     | trialmcp-dicom   |     | trialmcp-dicom   |
| -> Orthanc       |     | -> dcm4chee      |     | -> Orthanc       |
| trialmcp-authz   |     | trialmcp-authz   |     | trialmcp-authz   |
| trialmcp-ledger  |     | trialmcp-ledger  |     | trialmcp-ledger  |
+--------|--------+     +--------|--------+     +--------|--------+
         |                        |                        |
         +----------- Harmonization Layer -----------------+
         |   LOINC | RxNorm | MedDRA | RECIST 1.1         |
         +------|------|------|------|---------------------+
                v      v      v      v
+-------------------------------------------------------------+
|          FEDERATED COORDINATION LAYER                       |
|   FedAvg / FedProx / SCAFFOLD Aggregation                  |
|   Cross-Site Audit Sync | Differential Privacy             |
+-------------------------------------------------------------+
         |
         v
+-------------------------------------------------------------+
|      trialmcp-provenance (Central Gateway)                  |
| Cross-Site Lineage | Data Fingerprinting | Actor History  |
+-------------------------------------------------------------+
```

Each site can use different FHIR and DICOM backends (HAPI vs. other FHIR servers, Orthanc vs. dcm4chee) while maintaining a consistent MCP interface to agents.

4 Discussion

4.1 Clean Boundaries Between Protocol Layers

TrialMCP enforces a three-layer separation:

Table 8: Three-layer protocol separation in TrialMCP

Layer	Protocol	Responsibility
Agent Layer	MCP	Tool discovery, invocation, response handling
Clinical Layer	FHIR R4 / DICOM	Data representation, query semantics, resource typing
Infrastructure	HTTP/DICOM Net	Transport, authentication, session management

Agents never need to understand FHIR query syntax or DICOM networking protocols. They interact with typed MCP tools that abstract the underlying clinical standards while preserving semantic fidelity.

4.2 Conformance Considerations for HL7 and IHE

TrialMCP's FHIR server aligns with several HL7 and IHE integration profiles [2]:

- **IHE MHD** (Mobile access to Health Documents): The read-only FHIR access pattern with resource-level de-identification aligns with MHD's document sharing model
- **IHE XDS-I.b** (Cross-Enterprise Document Sharing for Imaging): The DICOM query/retrieve gating pattern maps to XDS-I.b's imaging document source/consumer actors
- **IHE ATNA** (Audit Trail and Node Authentication): The hash-chained audit ledger extends ATNA's audit logging with tamper-evident chain integrity
- **HL7 FHIR US Core**: Patient, Condition, Observation, and MedicationAdministration resources follow US Core profiles where applicable

4.3 Predictable Interfaces with Audit Hooks

Every MCP tool call produces a predictable response envelope with consistent error codes. The shared error taxonomy (`servers/common`) ensures that all 5 servers return the same error structure:

Standardized Error Response (v0.2.0)

```
# Success response
{"resource": {...}}

# Error response
{
    "error": "Invalid resource ID: http://evil.com",
    "error_code": "VALIDATION_FAILED"
}

# Authorization error
{
    "decision": "deny",
    "error_code": "AUTHZ_DENIED",
    "rule_id": "deny-provenance-write",
    "trace": [{"rule_id": "...", "effect": "deny"}]
}
```

4.4 Reduced Integration Complexity at Scale

The quantitative impact of MCP-based interoperability grows with trial scale:

Table 9: Integration complexity scaling comparison

Trial Scale	Point-to-Point	TrialMCP	Reduction
2 robots, 4 systems	8	6	25%
3 robots, 6 systems	18	9	50%
5 robots, 6 systems	30	11	63%
7 robots, 6 systems	42	13	69%
10 robots, 8 systems	80	18	78%

4.5 Agentic AI Pattern Integration

TrialMCP's MCP server design is informed by six agentic AI patterns from the federated learning reference architecture [3]:

1. **MCP Oncology Server**: Five core tools pattern implemented across all servers
2. **ReAct Treatment Planner**: Thought-action-observation loops for clinical decisions
3. **Real-Time Adaptive Monitoring**: Streaming data with CTCAE v5.0 grading
4. **Autonomous Simulation Orchestrator**: Multi-framework job coordination
5. **Safety-Constrained Executor**: IEC 80601-2-77 safety gates
6. **Oncology RAG Compliance**: Regulatory document retrieval with citation tracking

These patterns inform the tool interface design: each MCP tool is a discrete, auditable operation that can be composed into larger agentic workflows while maintaining complete traceability.

5 Limitations and Future Work

5.1 Current Limitations

1. **In-memory data stores**: Production deployment requires HAPI FHIR, Orthanc/dcm4chee, and persistent database backends. The current in-memory stores are suitable for development and testing only.
2. **No DICOMweb support**: The DICOM server uses a custom query interface. Production deployments should support both DICOM networking (C-FIND/C-MOVE) and DICOMweb (WADO-RS, QIDO-RS, STOW-RS).
3. **Limited FHIR search**: The FHIR search implementation uses simple string matching. Production deployments should support FHIR search parameter syntax including chaining, includes, and modifiers.
4. **No FHIR Subscription support**: Real-time event notification (e.g., new adverse event) is not implemented. FHIR R4 Subscription or R5 SubscriptionTopic should be considered.
5. **No HL7 v2 support**: Legacy HL7 v2 messaging interfaces (ADT, ORM, ORU) are not supported. Many hospital systems still rely on HL7 v2 for real-time integration.
6. **No IHE conformance testing**: The alignment with IHE profiles is architectural; formal IHE Connectathon testing has not been conducted.

5.2 Future Interoperability Work

- **DICOMweb proxy**: WADO-RS, QIDO-RS, and STOW-RS support alongside DICOM networking
- **FHIR Bulk Data**: Support for FHIR Bulk Data Access (SMART Backend Services) for large-scale data export
- **SMART on FHIR**: OAuth2-based app launch for EHR-integrated agent deployment
- **HL7 v2 gateway**: ADT/ORM/ORU message translation for legacy system integration
- **IHE Connectathon**: Formal conformance testing against IHE integration profiles
- **W3C PROV**: Alignment of the provenance model with W3C PROV-DM for cross-system interoperability
- **FHIR R5 migration**: Adoption of FHIR R5 features including SubscriptionTopic and improved search

6 Conclusion

TrialMCP positions the Model Context Protocol as the agent-tool interface that cleanly composes with FHIR R4 and DICOM in federated, multi-site oncology trial environments. The five-server architecture with 23 tools provides predictable, typed interfaces with audit hooks at every boundary. By separating the agent protocol layer (MCP) from the clinical data layer (FHIR/DICOM) and the infrastructure layer (HTTP/DICOM networking), TrialMCP enables agents to interact with clinical systems without understanding the underlying healthcare standards.

The architecture reduces integration complexity from $O(N \times M)$ to $O(N + M)$, provides clean boundaries between protocol layers, and supports federated deployment with per-site data sovereignty. Implementation pathways through HAPI FHIR, Orthanc, and dcm4chee provide concrete production deployment options. For health IT architects, interoperability engineers, and standards contributors, TrialMCP v0.2.0 demonstrates that MCP can serve as a principled interoperability layer for Physical AI agents in regulated clinical environments, complementing rather than replacing existing FHIR and DICOM infrastructure.

References

[1] HL7 International. FHIR R4 specification (v4.0.1). HL7, 2019. Fast Healthcare Interoperability Resources Release 4.

[2] IHE International. IHE integration profiles. IHE, 2024. Integrating the Healthcare Enterprise profiles.

[3] Kevin Kawchak. Federated learning for PAI oncology trials. GitHub, 2026. Five-pillar architecture, six agentic AI patterns.

[4] Linux Foundation AAIF. Model context protocol specification. Linux Foundation, 2025. Open protocol for connecting AI tools to external data sources.

[5] NEMA. DICOM standard. NEMA, 2024. Digital Imaging and Communications in Medicine.

[6] Orthanc. Orthanc – open-source DICOM server. Orthanc, 2024. Lightweight open-source DICOM server.

[7] Smile CDR. HAPI FHIR – open source FHIR server. GitHub, 2024. Java-based open-source FHIR server.

Acknowledgments

The author would like to acknowledge Anthropic for providing access to Claude Code Opus 4.6 for repository and paper generations; and OpenAI for providing access to GPT-5.2-Codex for peer-review recommendations, with ChatGPT 5.2 Thinking for project formulation and paper perspectives assistance.

Ethical Disclosures

The author of the article declares no competing interests.

Rights and Permissions

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author(s) and source are properly credited, a link to the Creative Commons license is provided, and any modifications made are indicated. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/.

Cite This Article

Kawchak K. TrialMCP: MCP Servers for Physical AI Oncology Clinical Trial Systems. *Zenodo*. 2026; 10.5281/zenodo.18870961.